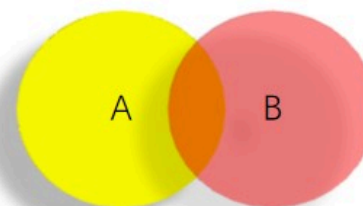


准备知识

Bayes 定理

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A)$$



当A为真的时候，B发生的可能性

先验概率

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

后验概率；B发生的时候，A发生的概率

B为真的先验概率

目录

- 线性回归
- 对数几率回归
- 线性判别分析
- 类别不平衡问题



准备知识

准备知识

◆ 高等数学：导数，梯度。

准备知识

◆ 高等数学：导数，梯度

实数： $x \in \mathbb{R}$ ，如 $x = 5$ 。

2维向量： $\mathbf{x} = (x_1, x_2) \in \mathbb{R}^2$ ，如 $\mathbf{x} = (3, 5)$ 。

n 维向量： $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ ，
如 $\mathbf{x} = (1, 2, \dots, n)$ 。

准备知识

◆ 高等数学：导数

设定义域和值域都在实数域中的一元函数 $y = f(x)$ 。当自变量 x 在 x_0 处取得增量 Δx 时，相应地应变量 y 取得增量 $\Delta y = f(x_0 + \Delta x) - f(x_0)$ 。如果极限 $\lim_{\Delta x \rightarrow 0} \frac{\Delta y}{\Delta x}$ 存在，则称函数 $y = f(x)$ 在 x_0 处可导，且导数为

$$f'(x_0) = \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}$$

准备知识

◆ 高等数学：导数

函数 $f(x)$

导数 $f'(x)$

$$x$$

$$1$$

$$x^2$$

$$2x$$

$$\frac{1}{x^2}$$

$$\frac{1}{2}x^{-\frac{1}{2}}$$

$$\sin x$$

$$\cos x$$

准备知识

◆ 高等数学：偏导数

一个多元函数 $y = f(x_1, \dots, x_m)$ 在点 $(a_1, \dots, a_m)$ 处对分量 x_1 的偏导数定义为

$$\frac{\partial f}{\partial x_1}(a_1, \dots, a_m) = \lim_{\Delta a \rightarrow 0} \frac{f(a_1 + \Delta a, \dots, a_m) - f(a_1, \dots, a_m)}{\Delta a}$$

简单记为 $f'_{x_1}(a_1, \dots, a_m)$

准备知识

◆ 高等数学：偏导数

函数 $f(x_1, x_2)$ 偏导数 $f'_{x_1}(x_1, x_2)$ 偏导数 $f'_{x_2}(x_1, x_2)$

$$x_1 + x_2$$

$$1$$

$$1$$

$$x_1^2 + x_2$$

$$2x_1$$

$$1$$

$$x_1^2 x_2$$

$$2x_1 x_2$$

$$x_1^2$$

$$\sin x_1 \cos x_2$$

$$\cos x_1 \cos x_2$$

$$-\sin x_1 \sin x_2$$

准备知识

◆ 高等数学：梯度

一个多元函数 $y = f(x_1, \dots, x_m)$ 在点 $\mathbf{a} = (a_1, \dots, a_m)$ 处的梯度定义为

$$\nabla f(\mathbf{a}) = \left(\frac{\partial f}{\partial x_1}(\mathbf{a}), \dots, \frac{\partial f}{\partial x_m}(\mathbf{a}) \right)$$

一般地，梯度 $\nabla f(\mathbf{a})$ 是一个向量。

准备知识

◆ 高等数学：梯度

函数 $f(x_1, x_2)$

梯度 $\nabla f(x_1, x_2)$

$$x_1 + x_2$$

$$(1, 1)$$

$$x_1^2 + x_2$$

$$(2x_1, 1)$$

$$x_1^2 x_2$$

$$(2x_1 x_2, x_1^2)$$

$$\sin x_1 \cos x_2$$

$$(\cos x_1 \cos x_2, -\sin x_1 \sin x_2)$$

问题描述

房价估计 (House Price Estimation)



图1：线性回归实例：房价估计。

问题描述

房价估计 (House Price Estimation)

面积(m^2)	房龄(年)	房间个数	房价(万)
136.54	2002	3	162
44.12	2000	1	60
$\vdots$	$\vdots$	$\vdots$	$\vdots$
90	2016	3	?

$$\mathbf{x}_1 = (x_{11}, x_{12}, \dots, x_{1d})^T$$

房子的不同属性

$$y_1$$

房价

问题描述

问题公式化

符号

含义

$\mathbf{x}_i \in \mathbb{R}^d$

第 i 个训练样本

$y_i \in \mathbb{R}$

第 i 个训练样本标记

$\mathbf{x} \in \mathbb{R}^d$

未知类别的测试样本

给定 n 个训练样本 $\mathbf{x}_1, \dots, \mathbf{x}_m$ 及其标记 $y_1, \dots, y_m$ ，线性回归问题的目标为学习一个线性模型准确估计任意新的样本 $\mathbf{x}$ 的输出标记。

问题描述

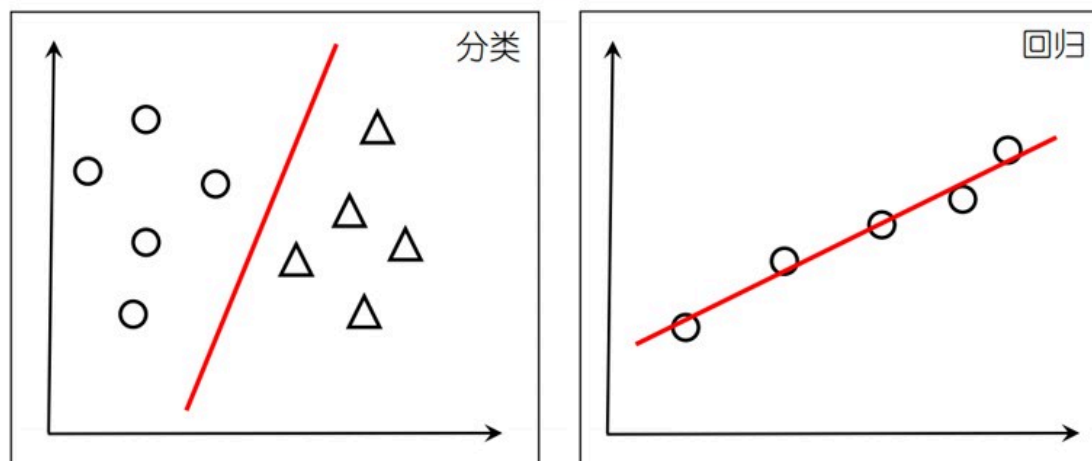
给定数据 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)$, 线性回归的目标为学习合适的参数 $\mathbf{w}$ 和 b 使得

$$y_i \approx f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b, i = 1, \dots, m \quad (1)$$

那么，如何找到参数 $\mathbf{w}$ 和 b 呢？

分类和回归

线性模型



线性模型(linear model)试图学得一个通过属性的线性组合来进行预测的函数

$$f(x) = w_1x_1 + w_2x_2 + \dots + w_dx_d + b$$

向量形式: $f(x) = w^T x + b$

简单、基本、可理解性好

16

线性回归--算法原理

◆ 先考虑一元线性回归问题，即特征维数 $d = 1$

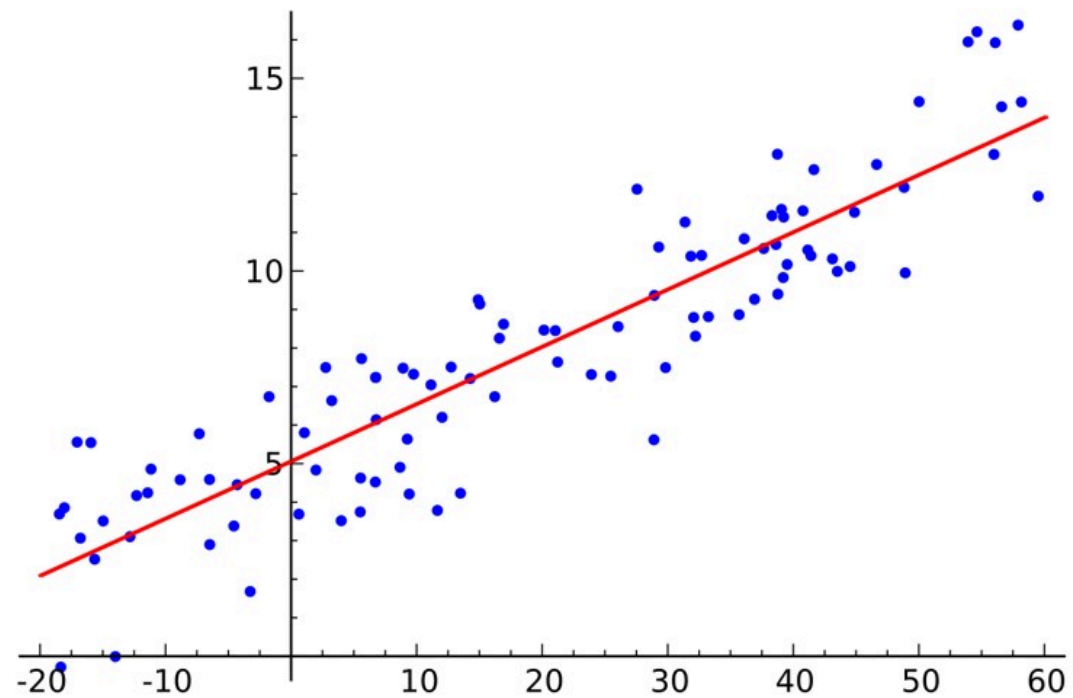
给定数据 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$, 线性回归的目标为学习合适的参数 w 和 b 使得

$$y_i \approx f(x_i) = wx_i + b, \quad i = 1, \dots, m \quad (1)$$

理想情况下：希望可以找到参数 w 和 b 使得

$$y_i = f(x_i) = wx_i + b, \quad i = 1, \dots, m$$

线性回归--算法原理



线性回归--算法原理

◆ 先考虑一元线性回归问题，即特征维数 $d = 1$

给定数据 $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$, 线性回归的目标为学习合适的参数 w 和 b 使得

$$y_i \approx f(x_i) = wx_i + b, \quad i = 1, \dots, m \quad (1)$$

但是如果不能找到合适的参数 w 和 b 使得等式(1)成立，那怎么办呢？

如何度量 y_i 和 $f(x_i)$ 的差异呢？

线性回归--算法原理

损失函数定义

均方误差 (Mean Square Error, MSE)

$$E(w, b) = \sum_{i=1}^m (y_i - f(x_i))^2$$

L1误差 (L1 norm Error)

$$E(w, b) = \sum_{i=1}^m |y_i - f(x_i)|$$

线性回归--算法原理

应该选哪种损失函数呢？为什么？

线性回归--算法原理

$$\begin{aligned}(w^*, b^*) &= \operatorname{argmin}_{(w, b)} \sum_{i=1}^m (y_i - f(x_i))^2 \\ &= \operatorname{argmin}_{(w, b)} \sum_{i=1}^m (wx_i + b - y_i)^2\end{aligned}$$

基于均方误差最小化来求解模型参数的方法称为**最小二乘法(Least Square method)**。

几何意义：最小二乘法试图找到一条直线，使得所有样本的预测误差平方和最小。

线性回归--算法原理

如何找到最小化如下目标函数的参数 (w, b) 呢?

$$E(w, b) = \sum_{i=1}^m (wx_i + b - y_i)^2$$

通过计算 $E(w, b)$ 对 w 和 b 的偏导, 并令其为零, 得到最优解。

线性回归--算法原理

$$\frac{\partial E(w, b)}{\partial w} = \sum_{i=1}^m 2x_i(wx_i + b - y_i) \quad (3.5)$$

$$\frac{\partial E(w, b)}{\partial b} = \sum_{i=1}^m 2(wx_i + b - y_i) \quad (3.6)$$

令式(3.5)和(3.6) 为零，计算最优解 w 和 b 。

线性回归--算法原理

最优解 w 和 b 为

$$w = \frac{\sum_{i=1}^m y_i(x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - m\bar{x}^2} = \frac{\sum_{i=1}^m y_i(x_i - \bar{x})}{\sum_{i=1}^m x_i^2 - \frac{1}{m}(\sum_{i=1}^m x_i)^2} \quad (3.7)$$

$$b = \bar{y} - \bar{x}w = \frac{1}{m} \sum_{i=1}^m (y_i - wx_i) \quad (3.8)$$

线性回归--算法原理

通过简单推导和整理，可重写式(3.5)和(3.6) 如下

$$(\sum_{i=1}^m x_i^2)w + (\sum_{i=1}^m x_i)b = \sum_{i=1}^m x_i y_i \quad (1)$$

$$(\sum_{i=1}^m x_i)w + mb = \sum_{i=1}^m y_i \quad (2)$$

(1)-(2)* $\bar{x}$ 即可去掉参数 b ，得到 w 的最优解。再由(2)即可得到参数 b 的最优解。

线性回归--算法原理

现在考虑多元线性回归问题，即特征维数 $d > 1$

给定数据 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)$

$$\mathbf{x}_1 = \begin{pmatrix} x_{11} \\ x_{12} \\ \vdots \\ x_{1d} \end{pmatrix} \in \mathbb{R}^d, \quad \dots, \quad \mathbf{x}_m = \begin{pmatrix} x_{m1} \\ x_{m2} \\ \vdots \\ x_{md} \end{pmatrix} \in \mathbb{R}^d$$

线性回归--算法原理

现在考虑多元线性回归问题，即特征维数 $d > 1$

给定数据 $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)$, 线性回归的目标为学习合适的参数 $\mathbf{w} \in \mathbb{R}^d$ 和 $b \in \mathbb{R}$ 使得

$$y_i \approx f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b, \quad i = 1, \dots, m \quad (1)$$

这称为**多元线性回归 (Multivariate Linear Regression)**

线性回归--算法原理

如何找到最小化如下目标函数的参数 $(\mathbf{w}, b)$ 呢?

$$\begin{aligned} E(\mathbf{w}, b) &= \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 \\ &= \sum_{i=1}^m \left(\sum_{j=1}^d w_j x_{ij} + b - y_i \right)^2 \end{aligned}$$

通过计算 $E(\mathbf{w}, b)$ 对 $w_1, \dots, w_d$ 和 b 的偏导, 并令其为零, 得到最优解。

线性回归--算法原理

$$\frac{\partial E(\mathbf{w}, b)}{\partial w_1} = \sum_{i=1}^m 2x_{i1} \left(\sum_{j=1}^d w_j x_{ij} + b - y_i \right)$$

⋮

$$\frac{\partial E(\mathbf{w}, b)}{\partial w_d} = \sum_{i=1}^m 2x_{id} \left(\sum_{j=1}^d w_j x_{ij} + b - y_i \right)$$



$$\frac{\partial E(\mathbf{w}, b)}{\partial b} = \sum_{i=1}^m 2 (\mathbf{w}^T \mathbf{x}_i + b - y_i)$$



令上述式子为零，可计算最优解 $\mathbf{w}$ 和 b 。

线性回归--算法原理

有没有更加**简单**的求解方法呢？

有，但是需要更高级的工具

矩阵微分

线性回归--算法原理

为便于计算，考虑如下定义

$$\hat{\mathbf{x}}_i = \begin{pmatrix} \mathbf{x}_i \\ 1 \end{pmatrix} = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \\ 1 \end{pmatrix} \in \mathbb{R}^{d+1} \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix} \in \mathbb{R}^m$$

$$\hat{\mathbf{w}} = \begin{pmatrix} \mathbf{w} \\ b \end{pmatrix} \in \mathbb{R}^{d+1}$$

线性回归--算法原理

$$\begin{aligned} E(\mathbf{w}, b) &= \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 \\ &= \sum_{i=1}^m (\hat{\mathbf{w}}^T \hat{\mathbf{x}}_i - y_i)^2 \end{aligned}$$

线性回归--算法原理

为便于计算，考虑如下定义

$$\mathbf{X} = \begin{pmatrix} \hat{\mathbf{x}}_1^T \\ \hat{\mathbf{x}}_2^T \\ \vdots \\ \hat{\mathbf{x}}_m^T \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix}$$

线性回归--算法原理

$$\begin{aligned} E(\mathbf{w}, b) &= \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 \\ &= \sum_{i=1}^m (\hat{\mathbf{w}}^T \hat{\mathbf{x}}_i - y_i)^2 \\ &= \|\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}\|_2^2 \\ &= (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}) \end{aligned}$$

线性回归--算法原理

因此，原问题可简化为

$$\hat{\mathbf{w}}^* = \underset{\hat{\mathbf{w}}}{\operatorname{argmin}} (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}) \quad (3.9)$$

定义 $E(\hat{\mathbf{w}}) = (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})$ ，求其梯度为

$$\nabla E(\hat{\mathbf{w}}) = 2\mathbf{X}^T (\mathbf{X}\hat{\mathbf{w}} - \mathbf{y})$$



这个是怎么得到的呢
?

线性回归--算法原理

令 $\nabla E(\hat{\mathbf{w}}) = 2\mathbf{X}^T(\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}) = 0$ ，可得到

$$\mathbf{X}^T\mathbf{X}\hat{\mathbf{w}} = \mathbf{X}^T\mathbf{y}$$

当 $\mathbf{X}^T\mathbf{X}$ 为满秩矩阵时，可得

$$\hat{\mathbf{w}}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

由于 $\hat{\mathbf{w}} = \begin{pmatrix} \mathbf{w} \\ b \end{pmatrix}$ ，因此

$$\mathbf{w}^* = \hat{\mathbf{w}}^*(1:d) \quad b^* = \hat{\mathbf{w}}^*(d+1)$$

线性回归--算法原理

令 $\nabla E(\hat{\mathbf{w}}) = 2\mathbf{X}^T(\mathbf{X}\hat{\mathbf{w}} - \mathbf{y}) = \mathbf{0}$ ，可得到

$$\mathbf{X}^T\mathbf{X}\hat{\mathbf{w}} = \mathbf{X}^T\mathbf{y}$$

当 $\mathbf{X}^T\mathbf{X}$ 不是满秩矩阵时，根据上述方程可得到多个 $\hat{\mathbf{w}}$ ，此时可以采用正则化方法。

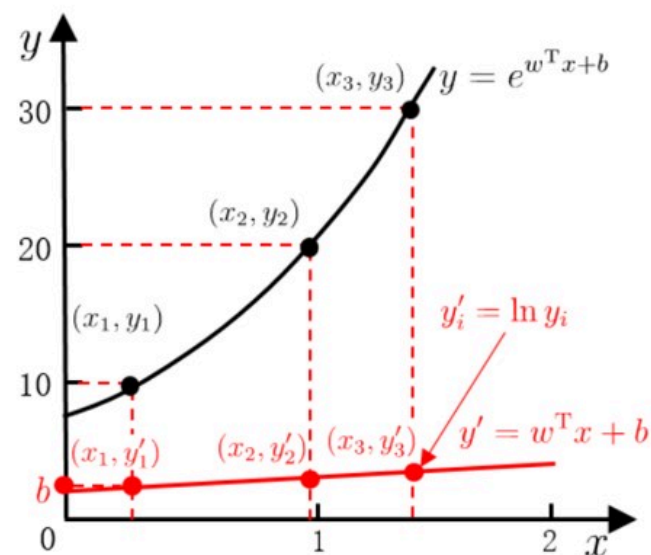
线性回归的变化

对于样例 (x, y) , $y \in \mathbb{R}$, 若希望线性模型的预测值逼近真实标记,
则得到线性回归模型 $y = \mathbf{w}^T \mathbf{x} + b$

令预测值逼近 y 的衍生物 ?

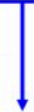
若令 $\ln y = \mathbf{w}^T \mathbf{x} + b$
则得到对数线性回归
(log-linear regression)

实际是在用 $e^{\mathbf{w}^T \mathbf{x} + b}$ 逼近 y



广义线性回归

一般形式: $y = g^{-1}(w^T x + b)$



单调可微的 **联系函数** (link function)

令 $g(\cdot) = \ln(\cdot)$ 则得到 对数线性回归

$$\ln y = w^T x + b$$

...

作业

1.如果把线性回归中的损失函数换成



$$E(\mathbf{w}, b) = \sum_{i=1}^m (\mathbf{w}^T \mathbf{x}_i + b - y_i)^2 + \lambda(\|\mathbf{w}\|_2^2 + b^2)$$

其中 $\lambda > 0$ 为常数。如何估计 $\mathbf{w}$ 和 b ?

注：上述问题等价于

$$E(\hat{\mathbf{w}}) = \sum_{i=1}^m (\hat{\mathbf{w}}^T \hat{\mathbf{x}}_i - y_i)^2 + \lambda \|\hat{\mathbf{w}}\|_2^2$$

其中 $\hat{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$, $\hat{\mathbf{x}}_i = \begin{bmatrix} \mathbf{x}_i \\ 1 \end{bmatrix}$, $i = 1, \dots, m$

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

```
>>> import numpy as np
>>> from sklearn.linear_model import LinearRegression
>>> X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
>>> #  $y = 1 * x_0 + 2 * x_1 + 3$ 
>>> y = np.dot(X, np.array([1, 2])) + 3
>>> reg = LinearRegression().fit(X, y)
>>> reg.score(X, y)
1.0
>>> reg.coef_
array([1., 2.])
>>> reg.intercept_
3.0...
>>> reg.predict(np.array([[3, 5]]))
array([16.])
```

42

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

▽ 案例：餐厅利润预测

假设你是一家餐厅的领导，正在考虑在不同的城市开设新的分店。该连锁店已经在不同的城市有了餐车，并且你能够获得每个城市的人口和利润数据。

现在需要使用这些数据来帮助你选择下一个被扩展的城市。

文件ex1data1.txt包含线性回归问题的数据集。第一列数据对应城市人口，第二列数据对应那座城市的餐车的利润。利润为负时表示亏损。

1. 读取并绘制数据

在开始进入实验之前，对数据进行可视化通常很有用。对于该数据集，可以使用散点图进行可视化，因为它只有两个属性（人口、利润）。

```
# 引入所需要的库文件
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from sklearn import linear_model
import os

%matplotlib inline
```



43

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

```
# 数据存储路径
path = 'ex1data1.txt'

# 读入相应的数据文件
data = pd.read_csv(path, header=None, names=['Population', 'Profit'])

# 查看数据的前五条
data.head(5)

# 获取数据列数
cols = data.shape[1]

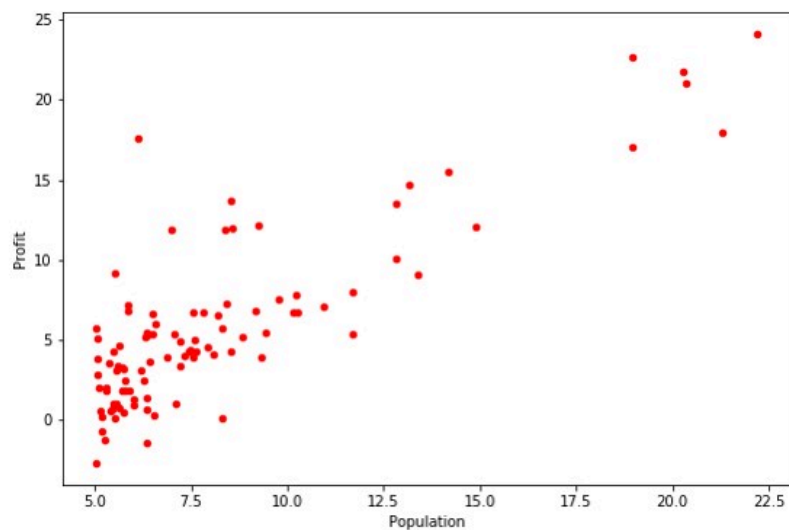
# 构建样本的特征和标记对
X = data.iloc[:, 0:cols-1]
y = data.iloc[:, cols-1:cols]
```

	Population	Profit
0	6.1101	17.5920
1	5.5277	9.1302
2	8.5186	13.6620
3	7.0032	11.8540
4	5.8598	6.8233

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

```
# 数据可视化  
data.plot(kind='scatter', x='Population', y='Profit', c='red', figsize=(9,6))
```



45

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

2. 训练模型

利用经验数据构建线性回归模型

```
# 利用数据学习线性回归模型
reg = linear_model.LinearRegression()
reg.fit(X,y)

# 打印模型系数和截距
print('Coefficients: \n', reg.coef_)
print('Intercept: \n', reg.intercept_)

✓ 0.9s
```

```
Coefficients:
[[1.19303364]]
Intercept:
[-3.89578088]
```

46

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

3. 预测新数据

利用学习到的模型预测任意新输入

```
# 预测任意新输入的标记，即在任意城市新开分店可能获得的利润
xtest=15
ytest_pred = reg.predict(xtest)
print('Predicted profit: \n', ytest_pred)
```

✓ 0.6s

Predicted profit:
[[13.99972378]]

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)

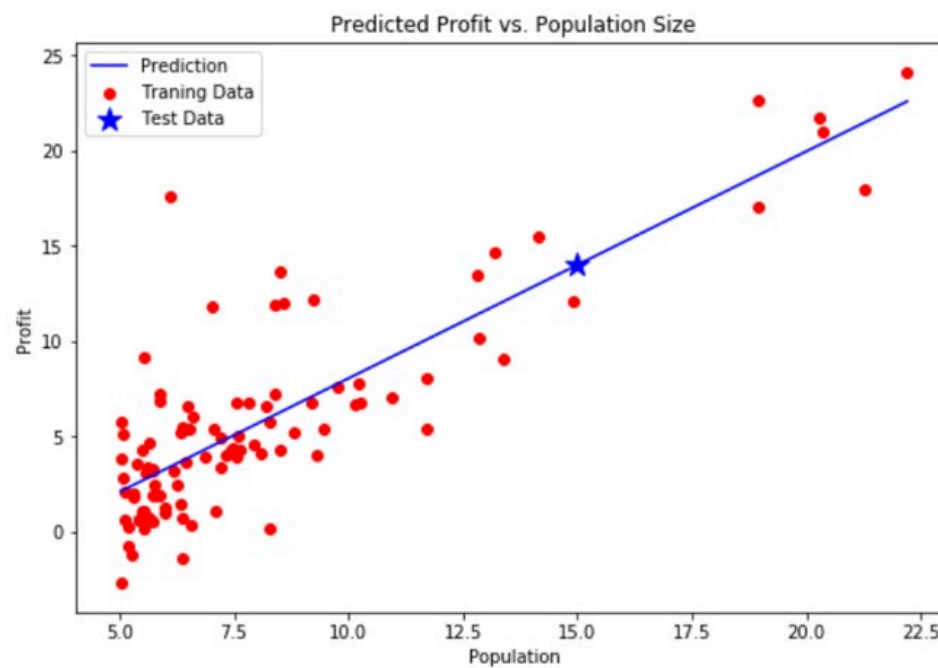
```
#对预测结果进行绘制
x = np.linspace(data.Population.min(), data.Population.max(), 100)
f = reg.coef_[0] * x + reg.intercept_

fig, ax = plt.subplots(figsize=(9,6))
ax.plot(x, f, 'b', label='Prediction')
ax.scatter(data.Population, data.Profit, c='red', label='Traning Data')
ax.scatter(xtest, ytest_pred, s=200, c='blue', marker='*', label='Test Data')
ax.legend(loc=2)
ax.set_xlabel('Population')
ax.set_ylabel('Profit')
ax.set_title('Predicted Profit vs. Population Size')
```

48

代码案例

◆ 调用线性回归代码 (采用Scikit-Learn)



49



对数几率回归

准备知识

- 射中的概率
 - $P(A)$: 0.6
 - $P(B)$: 0.5

- 目标被射中:
 - 被A击中
 - 被B击中



- 它被A击落的概率有多大?
 - C : 目标被击中.

$$P(A | C) = \frac{P(C | A)P(A)}{P(C)} = \frac{1 \times 0.6}{0.6 \times 0.5 + 0.4 \times 0.5 + 0.6 \times 0.5} = \frac{3}{4}$$

准备知识

- ◆ 概率统计：极大似然法
(Maximum Likelihood)
- ◆ 优化理论：梯度下降
(Gradient Descent)

极大似然法

一枚硬币，抛了**10次**，都是正面朝上，
你觉得可能吗？

如果抛了**100次**，仍然都是正面朝上，
你觉得可能吗？

很有可能这个硬币有问题！

极大似然法

设扔一次硬币，正面朝上的概率为参数 θ 。

一般来说，参数 θ 受硬币的材质，均匀程度等因素共同决定。反过来，参数 θ 也刻画了硬币的一些特性。



极大似然法

➤ 概率与似然 (Probability vs. Likelihood)

相同点：都是指可能性。

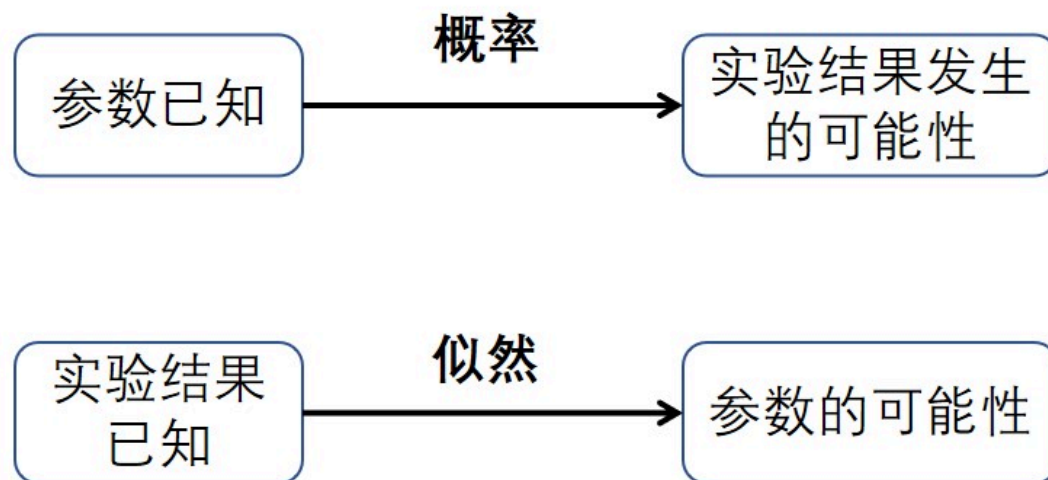
不同点：

已知硬币的参数，就可以去推测抛硬币的各种情况的可能性，这称为**概率**。

已知抛硬币的试验结果，推测硬币参数的可能性，这称为**似然**。

极大似然法

➤ 概率与似然 (Probability vs. Likelihood)



57

极大似然法

➤ 概率与似然 (Probability vs. Likelihood)

已知硬币参数为0.5，即硬币两面都是均匀的。请计算抛10次硬币，5次正面朝上的概率是多少？

$$P = C_{10}^5 * 0.5^5 * (1 - 0.5)^5 \approx 0.25$$

极大似然法

➤ 概率与似然 (Probability vs. Likelihood)

反过来，如果抛10次硬币，6次正面朝上。
请问硬币的参数取哪种值可能性最大呢？

如果参数为0.5，出现该实验结果的概率为

$$P = C_{10}^6 * 0.5^6 * (1 - 0.5)^4 \approx 0.21$$

如果参数为0.6，出现该实验结果的概率为

$$P = C_{10}^6 * 0.6^6 * (1 - 0.6)^4 \approx 0.25$$

极大似然法

➤ 概率与似然 (Probability vs. Likelihood)

反过来，如果抛10次硬币，6次正面朝上。
请问硬币的参数取哪种值可能性最大呢？

如果参数为 θ ，出现该实验结果的概率为

$$P = C_{10}^6 * \theta^6 * (1 - \theta)^4$$

寻求参数 θ ，使得上述概率值最大。

极大似然法

➤ 似然函数 (Likelihood Function)

◆ 离散随机变量:

设 X 为一个离散随机变量, 依赖参数 θ

$$L(\theta|x) = p_{\theta}(x) = P_{\theta}(X = x)$$

◆ 连续随机变量:

设 X 为一个连续随机变量, 依赖参数 θ

$$L(\theta|x) = f_{\theta}(x)$$

极大似然法

➤ 似然函数 (Likelihood Function)

◆ 离散随机变量:

如果有 m 个相互独立的观测值 $x_1, x_2, \dots, x_m$,
则其似然函数为

$$L(\theta|x_1, \dots, x_m) = p_\theta(x_1) \cdots p_\theta(x_m)$$

取对数后

$$\ln L(\theta|x_1, \dots, x_m) = \sum_{i=1}^m \ln p_\theta(x_i)$$

62

极大似然法

➤ 极大似然法 (Maximum Likelihood Method)

$$\begin{aligned}\theta^* &= \operatorname{argmax}_{\theta} L(\theta|x_1, \dots, x_m) \\ &= \operatorname{argmax}_{\theta} \ln L(\theta|x_1, \dots, x_m)\end{aligned}$$

极大似然法

极大似然法例子：

其实我们每天都在做极大似然法。

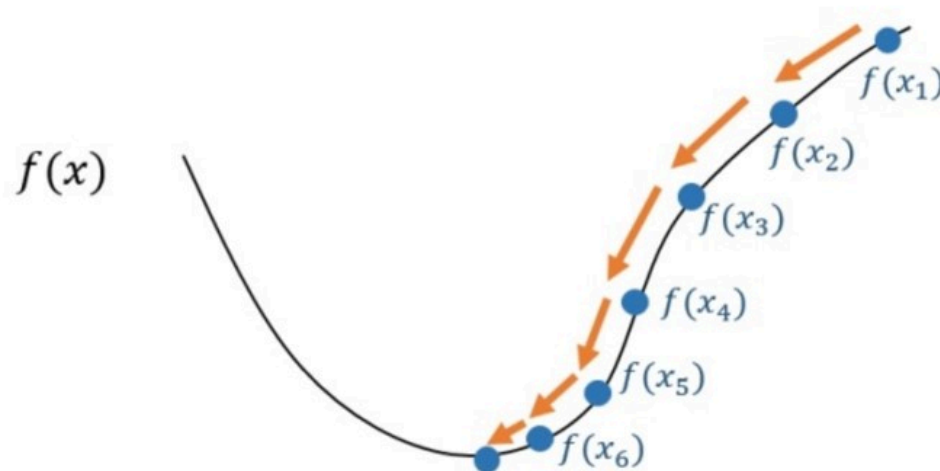
- 前方有个长发及腰的背影，你觉得他/她是？
- 老猎人和新菜鸟同时捕猎，一枪命中猎物，你觉得是谁的枪命中的？
- 公安局长的儿子跑过来说，你的爸爸和我的爸爸在吵架。请问这两个爸爸是什么关系？

64

梯度下降法

梯度下降法 (Gradient Descent)

梯度下降法是一种常用的一阶优化方法，被广泛应用于各种算法，尤其是深度神经网络算法。



65

梯度下降法

梯度下降法 (Gradient Descent)

➤ 考虑最简单的一元函数情形: $f(x)$, $x \in \mathbb{R}$

如何求解如下无约束优化问题:

$$\min_x f(x)$$

如果 $f(x)$ 可导, 通过求其导数 $f'(x)$, 并使其为0, 得到极值点。

但是如果方程 $f'(x) = 0$ 难以求解或者非常耗时, 怎么办呢?

66

梯度下降法

梯度下降法 (Gradient Descent)

- 一种可能的思路：如果能找到一个序列 $x_0, x_1, x_2, \dots$ 使得

$$f(x_{t+1}) < f(x_t), \quad t = 0, 1, \dots$$

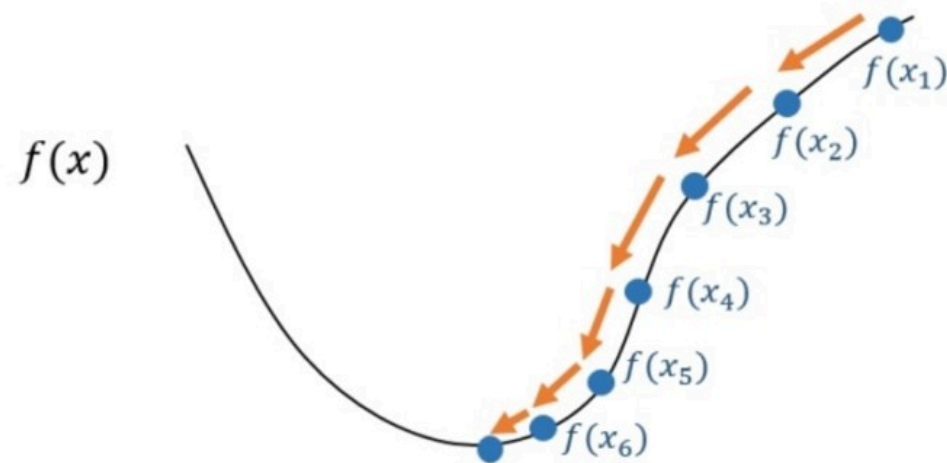
则通过迭代可找到 $f(x)$ 的局部极小点。

如果 $f(x)$ 为凸函数，则该局部极小点为 $f(x)$ 的最小值点。

梯度下降法

梯度下降法 (Gradient Descent)

➤ 一种可能的思路：



68

梯度下降法

梯度下降法 (Gradient Descent)

- 那么问题转化为：给定当前点 x_t ，如何找到下一个点 x_{t+1} ，使得

$$f(x_{t+1}) < f(x_t)$$

一定成立呢？

每次都左右看并且试探？

梯度下降法

梯度下降法 (Gradient Descent)

对于一元函数来说, x 的变化只有两个方向: 要么向右 ($\Delta x > 0$), 要么向左 ($\Delta x < 0$)

回忆一下高等数学中的泰勒公式:

$$f(x + \Delta x) = f(x) + \frac{f'(x)}{1!} \Delta x + \frac{f^{(2)}(x)}{2!} \Delta x^2 + \dots + \frac{f^{(n)}(x)}{n!} \Delta x^n + o(\Delta x^n)$$

$$f(x + \Delta x) = f(x) + f'(x)\Delta x + o(\Delta x)$$

梯度下降法

梯度下降法 (Gradient Descent)

当 Δx 的绝对值较小时, 我们有

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x$$

为了使得 $f(x + \Delta x) < f(x)$, 我们需要保证

$$f'(x)\Delta x = \Delta x \nabla f(x) < 0$$

当 $f(x)$ 为一元函数时, $\nabla f(x) = f'(x)$

71

梯度下降法

梯度下降法 (Gradient Descent)

那怎么选 Δx 呢?

72

梯度下降法

梯度下降法 (Gradient Descent)

可选择

$$\Delta x = -\eta \nabla f(x)$$

其中 η 称为步长, 为一个较小的正数 (为什么?)

则

$$\Delta x \nabla f(x) = -\eta (\nabla f(x))^2$$

- ◆ 如果 $\nabla f(x) \neq 0$, 则 $\Delta x \nabla f(x) < 0$
- ◆ 如果 $\nabla f(x) = 0$, 则当前点 x 为极值点

梯度下降法

梯度下降法 (Gradient Descent)

因此，通过设定 $\Delta x = -\eta \nabla f(x)$ 可保证

$$f(x + \Delta x) < f(x)$$

回到梯度下降法，更新公式为

$$x_{t+1} = x_t - \eta \nabla f(x_t)$$

问题描述

模式分类问题的目标为：设计智能算法自动将新的测试样本准确划分到真实的类别。

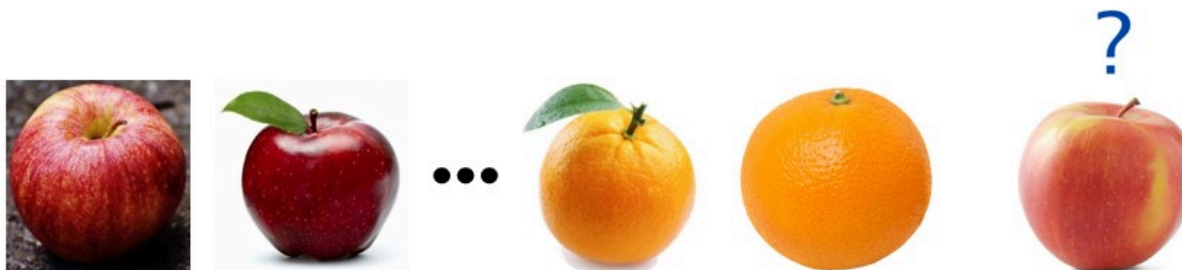


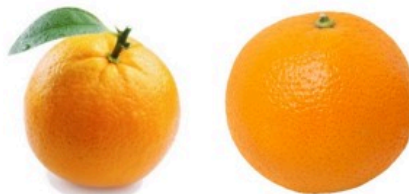
图1：模式分类的应用：水果识别，即给定多个类别水果的图像，准确识别任意新的未知类别的水果。

问题描述

训练样本
Training sample



标记为1



标记为0

测试样本
Testing sample

1 还是 0

?



问题描述



$(\mathbf{x}_1, y_1)$

特征：例如，

- 重量，长度，颜色，...
- 原始图像矩阵
- PCA特征，LBP特征，...

类别标记：1



$(\mathbf{x}_2, y_2)$

类别标记：0

问题描述



...

$(\mathbf{x}_1, y_1)$

$(\mathbf{x}_2, y_2)$

$(\mathbf{x}_3, y_3)$

$(\mathbf{x}_4, y_4)$

...

?



$(\mathbf{x}, ?)$

问题描述

问题公式化

符号	含义
2	类别数
m	训练样本个数
$\mathbf{x}_i$	第 i 个训练样本特征
$\mathbf{x} \in \mathbb{R}^d$	未知类别的测试样本

给定来自2类的 m 个训练样本 $\mathbf{x}_1, \dots, \mathbf{x}_m$ ，模式分类问题的目标为准确估计任意新的测试样本 $\mathbf{x}$ 的真实类别。

对数几率回归--算法原理

线性回归模型产生的实值输出 $z = \mathbf{w}^T \mathbf{x} + b$
期望输出 $y \in \{0, 1\}$

找 z 和 y 的联系函数

理想的“单位阶跃函数”
(unit-step function)

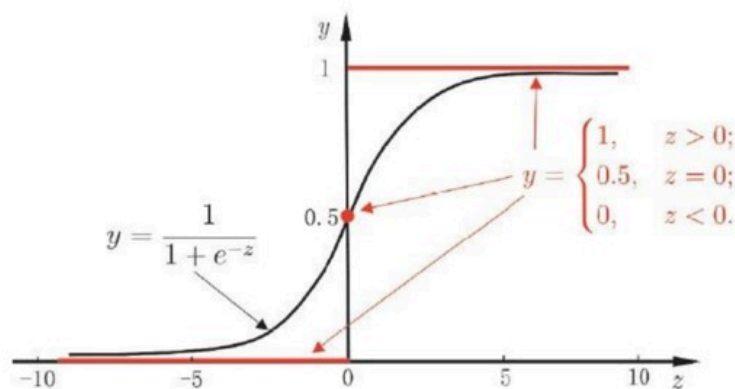
$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$

性质不好,
需找“替代函数”
(surrogate function)

常用
单调可微、任意阶可导

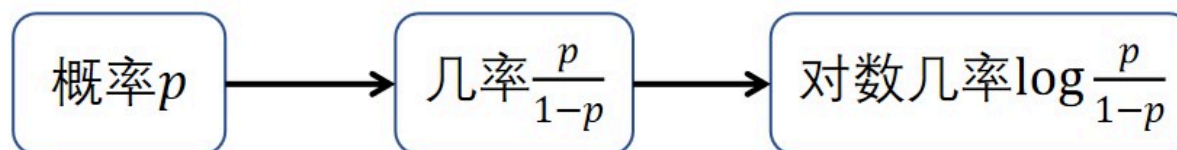
$$y = \frac{1}{1 + e^{-z}}$$

对数几率函数
(logistic function)
简称“对率函数”



对数几率回归--算法原理

- 概率、几率与对数几率 (Probability vs. Odds vs. Logit)



概率(Probability): 事件发生的可能性

几率(Odds): 事件发生和不发生的比率

对数几率(Logit): 几率取对数

对数几率回归--算法原理

➤ 对数几率回归模型

$$\ln \frac{p(y=1|\mathbf{x})}{p(y=0|\mathbf{x})} = \mathbf{w}^T \mathbf{x} + b, \quad (3.22)$$

为什么
?

由上式可得

$$p(y = 1|\mathbf{x}) = \frac{e^{\mathbf{w}^T \mathbf{x} + b}}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}, \quad (3.23)$$

$$p(y = 0|\mathbf{x}) = \frac{1}{1 + e^{\mathbf{w}^T \mathbf{x} + b}}, \quad (3.24)$$

对数几率回归--算法原理

➤ 似然函数

$$\ell(\mathbf{w}, b) = \sum_{i=1}^m \ln p(y_i | \mathbf{x}_i; \mathbf{w}, b) \quad (3.25)$$

为什么？

其中

$$p(y_i | \mathbf{x}_i; \mathbf{w}, b) = y_i p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) + (1 - y_i) p_0(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) \quad (3.26)$$

$$p_1(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) = p(y = 1 | \hat{\mathbf{x}}_i; \boldsymbol{\beta}), \quad \boldsymbol{\beta} = [\mathbf{w}, b]^T$$

$$p_0(\hat{\mathbf{x}}_i; \boldsymbol{\beta}) = p(y = 0 | \hat{\mathbf{x}}_i; \boldsymbol{\beta})$$

对数几率回归--算法原理

➤ 最小化目标函数

最大化似然函数等价于最小化

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^m (-y_i \boldsymbol{\beta}^T \hat{\mathbf{x}}_i + \ln(1 + e^{\boldsymbol{\beta}^T \hat{\mathbf{x}}_i})) \quad (3.27)$$

上式为关于的高阶可导连续凸函数，可采用梯度下降法，牛顿法等经典优化方法求解。

如何得到
?

为什么
?

对数几率回归--算法原理

➤ 分类

获得最优解后 $\beta^* = [\mathbf{w}^*, b^*]^T$ ，如何对测试样本 $\mathbf{x} \in \mathbb{R}^d$ 进行分类？

分别计算测试样本 $\mathbf{x}$ 属于第0类和第1类的概率

$$p(y = 1|\mathbf{x}) = \frac{e^{\mathbf{w}^{*T}\mathbf{x}+b^*}}{1+e^{\mathbf{w}^{*T}\mathbf{x}+b^*}},$$

$$p(y = 0|\mathbf{x}) = \frac{1}{1+e^{\mathbf{w}^{*T}\mathbf{x}+b^*}},$$

若 $p(y = 1|\mathbf{x}) > p(y = 0|\mathbf{x})$ ，则将 $\mathbf{x}$ 分到第1类；反之，分到第0类。

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

案例：疾病预测

假设你是一位医学数据分析师，假设给定一个病人的多种身体指标，如何利用Logistic Regression模型预测该病人是否患有某种疾病。

1. 读取数据

首先，从sklearn中读取原始数据

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score

# 加载 sklearn 自带的乳腺癌（分类）数据集
X, y = load_breast_cancer(return_X_y=True)

# 以指定比例将数据集分为训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, test_size=0.3)
```



86

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

案例：疾病预测

假设你是一位医学数据分析师，假设给定一个病人的多种身体指标，如何利用Logistic Regression模型预测该病人是否患有某种疾病。

sklearn.datasets.load_breast_cancer

```
sklearn.datasets.load_breast_cancer(*, return_X_y=False, as_frame=False)
```

[\[source\]](#)

Load and return the breast cancer wisconsin dataset (classification).

The breast cancer dataset is a classic and very easy binary classification dataset.

Classes	2
Samples per class	212(M),357(B)
Samples total	569
Dimensionality	30
Features	real, positive

The copy of UCI ML Breast Cancer Wisconsin (Diagnostic) dataset is downloaded from: <https://goo.gl/U2Uwz2>

Read more in the [User Guide](#).

Parameters: return_X_y : bool, default=False

If True, returns (data, target) instead of a Bunch object. See below for more information about the data and target object.

New in version 0.18.

as_frame : bool, default=False

If True, the data is a pandas DataFrame including columns with appropriate dtypes (numeric). The target is a pandas DataFrame or Series depending on the number of target columns. If return_X_y is True, then (data, target) will be pandas DataFrames or Series as described below.

New in version 0.23.

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html



87

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

案例：疾病预测

假设你是一位医学数据分析师，假设给定一个病人的多种身体指标，如何利用Logistic Regression模型预测该病人是否患有某种疾病。

1. 读取数据

首先，从sklearn中读取原始数据

```
print(X[0:2,:])
print(y[0:2])
```

✓ 0.5s

```
[[1.799e+01 1.038e+01 1.228e+02 1.001e+03 1.184e-01 2.776e-01 3.001e-01
 1.471e-01 2.419e-01 7.871e-02 1.095e+00 9.053e-01 8.589e+00 1.534e+02
 6.399e-03 4.904e-02 5.373e-02 1.587e-02 3.003e-02 6.193e-03 2.538e+01
 1.733e+01 1.846e+02 2.019e+03 1.622e-01 6.656e-01 7.119e-01 2.654e-01
 4.601e-01 1.189e-01]
[2.057e+01 1.777e+01 1.329e+02 1.326e+03 8.474e-02 7.864e-02 8.690e-02
 7.017e-02 1.812e-01 5.667e-02 5.435e-01 7.339e-01 3.398e+00 7.408e+01
 5.225e-03 1.308e-02 1.860e-02 1.340e-02 1.389e-02 3.532e-03 2.499e+01
 2.341e+01 1.588e+02 1.956e+03 1.238e-01 1.866e-01 2.416e-01 1.860e-01
 2.750e-01 8.902e-02]]
[0 0]
```



88

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

案例：疾病预测

假设你是一位医学数据分析师，假设给定一个病人的多种身体指标，如何利用Logistic Regression模型预测该病人是否患有某种疾病。

2. 训练模型

利用经验数据构建对数几率回归模型

```
# 利用训练数据来学习分类模型
clf = LogisticRegression(C=1.0)
clf.fit(X_train, y_train)
```

3. 预测新数据

利用学习到的模型预测任意新输入

```
# 使用测试数据来预测，返回值预测分类数据
y_pred = clf.predict(X_test)
```

89

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

sklearn.linear_model.LogisticRegression ¶

```
class sklearn.linear_model.LogisticRegression(penalty='l2', *, dual=False, tol=0.0001, C=1.0, fit_intercept=True,
intercept_scaling=1, class_weight=None, random_state=None, solver='lbfgs', max_iter=100, multi_class='auto', verbose=0,
warm_start=False, n_jobs=None, l1_ratio=None)
```

[source]

Logistic Regression (aka logit, MaxEnt) classifier.

In the multiclass case, the training algorithm uses the one-vs-rest (OvR) scheme if the 'multi_class' option is set to 'ovr', and uses the cross-entropy loss if the 'multi_class' option is set to 'multinomial'. (Currently the 'multinomial' option is supported only by the 'lbfgs', 'sag', 'saga' and 'newton-cg' solvers.)

This class implements regularized logistic regression using the 'liblinear' library, 'newton-cg', 'sag', 'saga' and 'lbfgs' solvers. **Note that regularization is applied by default.** It can handle both dense and sparse input. Use C-ordered arrays or CSR matrices containing 64-bit floats for optimal performance; any other input format will be converted (and copied).

The 'newton-cg', 'sag', and 'lbfgs' solvers support only L2 regularization with primal formulation, or no regularization. The 'liblinear' solver supports both L1 and L2 regularization, with a dual formulation only for the L2 penalty. The Elastic-Net regularization is only supported by the 'saga' solver.

Read more in the User Guide.

Parameters: **penalty** : {'l1', 'l2', 'elasticnet', None}, default='l2'
Specify the norm of the penalty:

- None: no penalty is added;
- 'l2': add a L2 penalty term and it is the default choice;
- 'l1': add a L1 penalty term;
- 'elasticnet': both L1 and L2 penalty terms are added.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression

90

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

1.1.11.1. Binary Case

For notational ease, we assume that the target y_i takes values in the set $\{0, 1\}$ for data point i . Once fitted, the `predict_proba` method of `LogisticRegression` predicts the probability of the positive class $P(y_i = 1|X_i)$ as

$$\hat{p}(X_i) = \text{expit}(X_i w + w_0) = \frac{1}{1 + \exp(-X_i w - w_0)}.$$

As an optimization problem, binary class logistic regression with regularization term $r(w)$ minimizes the following cost function:

$$\min_w C \sum_{i=1}^n (-y_i \log(\hat{p}(X_i)) - (1 - y_i) \log(1 - \hat{p}(X_i))) + r(w).$$

We currently provide four choices for the regularization term $r(w)$ via the `penalty` argument:

penalty	$r(w)$
None	0
ℓ_1	$\ w\ _1$
ℓ_2	$\frac{1}{2} \ w\ _2^2 = \frac{1}{2} w^T w$
ElasticNet	$\frac{1-\rho}{2} w^T w + \rho \ w\ _1$

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression

91

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

案例：疾病预测

假设你是一位医学数据分析师，假设给定一个病人的多种身体指标，如何利用Logistic Regression模型预测该病人是否患有某种疾病。

4. 评价预测结果

计算预测结果的多种评价指标

```
# 打印主要分类指标的文本报告
print('--- report ---')
print(classification_report(y_test, y_pred))

# 打印 auc
print('--- auc ---')
print(roc_auc_score(y_test, y_pred))
```

```
--- report ---
              precision    recall  f1-score   support

     0               0.97       0.93       0.95         70
     1               0.95       0.98       0.97        101

 avg / total          0.96       0.96       0.96        171

--- auc ---
0.9543847241867044
```

92

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

1.1.11.2. Multinomial Case

The binary case can be extended to K classes leading to the multinomial logistic regression, see also [log-linear model](#).

Note: It is possible to parameterize a K -class classification model using only $K - 1$ weight vectors, leaving one class probability fully determined by the other class probabilities by leveraging the fact that all class probabilities must sum to one. We deliberately choose to overparameterize the model using K weight vectors for ease of implementation and to preserve the symmetrical inductive bias regarding ordering of classes, see [16]. This effect becomes especially important when using regularization. The choice of overparameterization can be detrimental for unpenalized models since then the solution may not be unique, as shown in [16].

Let $y_i \in 1, \dots, K$ be the label (ordinal) encoded target variable for observation i . Instead of a single coefficient vector, we now have a matrix of coefficients W where each row vector W_k corresponds to class k . We aim at predicting the class probabilities $P(y_i = k | X_i)$ via `predict_proba` as:

$$\hat{p}_k(X_i) = \frac{\exp(X_i W_k + W_{0,k})}{\sum_{l=0}^{K-1} \exp(X_i W_l + W_{0,l})}.$$

The objective for the optimization becomes

$$\min_W -C \sum_{i=1}^n \sum_{k=0}^{K-1} [y_i = k] \log(\hat{p}_k(X_i)) + r(W).$$

Where $[P]$ represents the Iverson bracket which evaluates to 0 if P is false, otherwise it evaluates to 1. We currently provide four choices for the regularization term $r(W)$ via the `penalty` argument:

penalty	$r(W)$
None	0
ℓ_1	$\ W\ _{1,1} = \sum_{i=1}^n \sum_{j=1}^K W_{i,j} $
ℓ_2	$\frac{1}{2} \ W\ _F^2 = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^K W_{i,j}^2$
ElasticNet	$\frac{1-\rho}{2} \ W\ _F^2 + \rho \ W\ _{1,1}$

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html#sklearn.linear_model.LogisticRegression

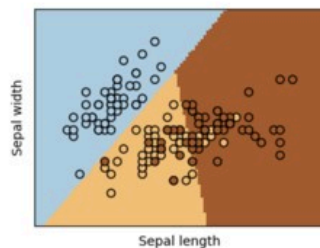
93

代码案例

◆ 对数几率回归使用代码 (采用Scikit-Learn)

Logistic Regression 3-class Classifier

Show below is a logistic-regression classifiers decision boundaries on the first two dimensions (sepal length and width) of the iris dataset. The datapoints are colored according to their labels.



```
# Code source: Gaël Varoquaux
# Modified for documentation by Jaques Grobler
# License: BSD 3 clause

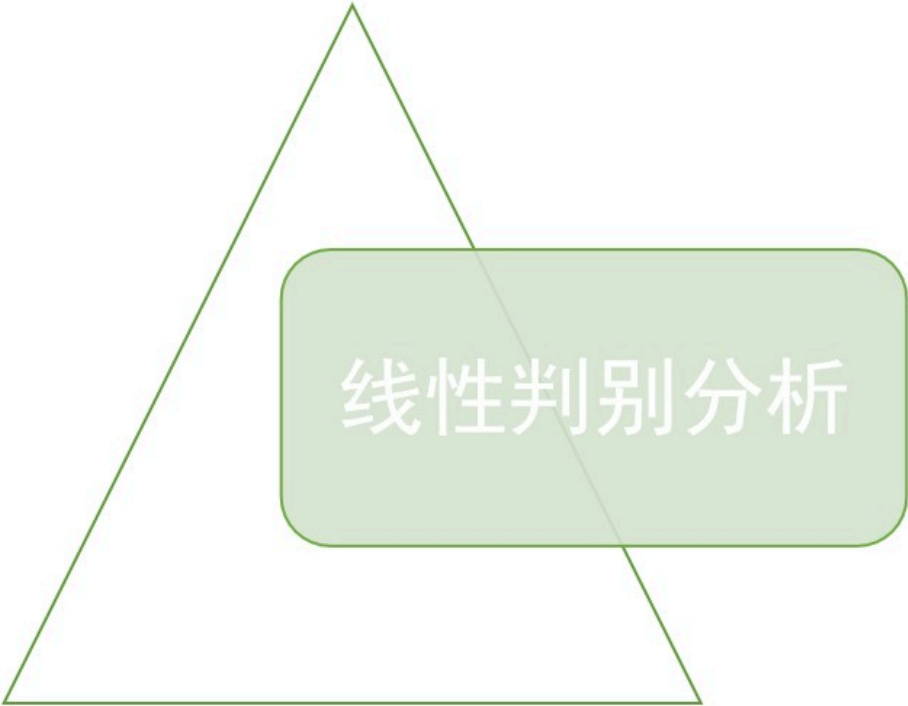
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn import datasets
from sklearn.inspection import DecisionBoundaryDisplay

# import some data to play with
iris = datasets.load_iris()
X = iris.data[:, :2] # we only take the first two features.
Y = iris.target

# Create an instance of Logistic Regression Classifier and fit the data.
logreg = LogisticRegression(C=1e5)
logreg.fit(X, Y)
```

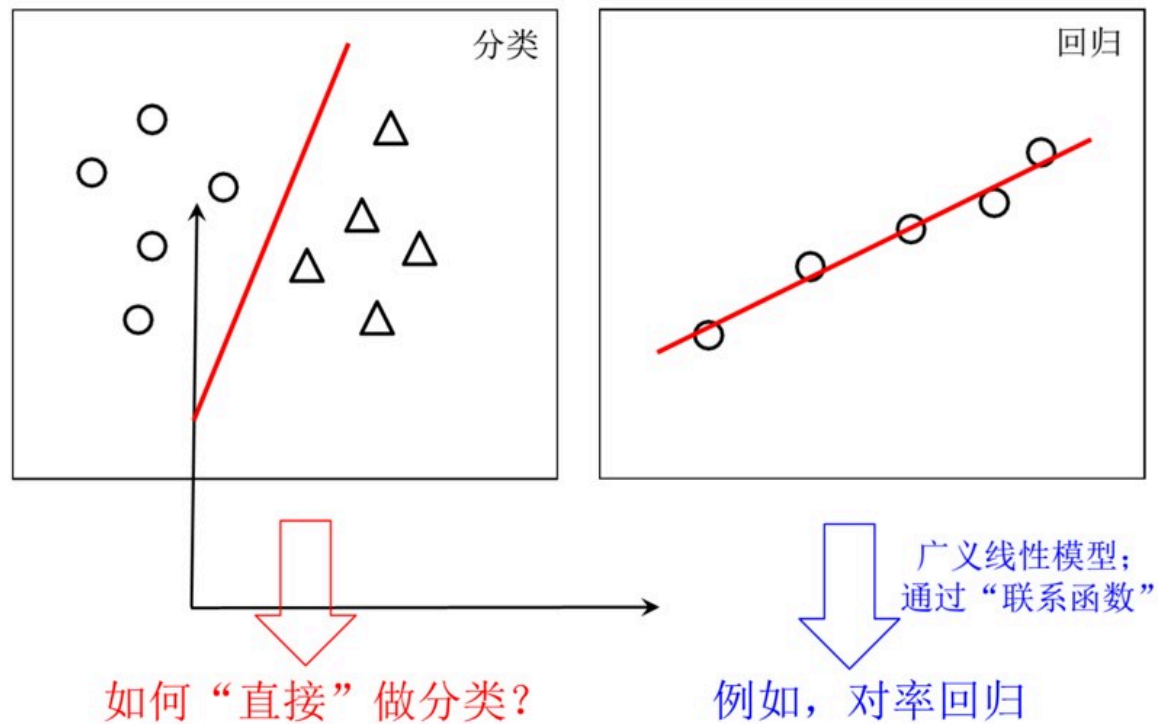
https://scikit-learn.org/stable/auto_examples/linear_model/plot_iris_logistic.html#

94



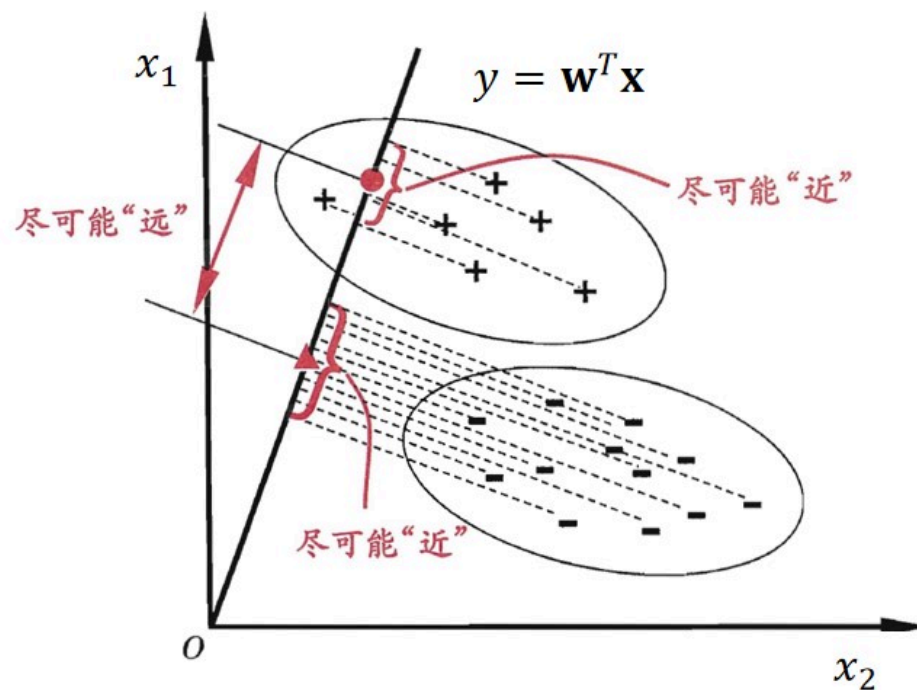
线性判别分析

线性判别分析--算法原理



线性判别分析--算法原理

线性判别分析(Linear Discriminant Analysis, LDA)



线性判别分析--算法原理

线性判别分析思想：寻找一个直线（或者低维子空间），使得同类样本的投影点尽可能接近，异类样本的投影点尽可能远离。

同类近，异类远。

如何量化“同类近，异类远”？

线性判别分析--算法原理

给定数据集 $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$

第 k 类样本的集合 X_k

第 k 类样本的均值向量 μ_i

投影向量 $\mathbf{w}$

样本 $\mathbf{x}_i$ 在向量上的投影 $\mathbf{w}^T \mathbf{x}_i$

线性判别分析--算法原理

同类样本的远近程度：

$$\sum_{\mathbf{x} \in X_0} \|\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_0\|_2^2$$

$$\sum_{\mathbf{x} \in X_1} \|\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_1\|_2^2$$

异类样本的远近程度：

$$\|\mathbf{w}^T \boldsymbol{\mu}_0 - \mathbf{w}^T \boldsymbol{\mu}_1\|_2^2$$

线性判别分析--算法原理

能否简化一下？

$$\sum_{\mathbf{x} \in X_0} \|\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_0\|_2^2 = \mathbf{w}^T \left(\sum_{\mathbf{x} \in X_0} (\mathbf{x} - \boldsymbol{\mu}_0)(\mathbf{x} - \boldsymbol{\mu}_0)^T \right) \mathbf{w}$$



协方差矩阵，记为 Σ_0 和 Σ_1

类似的，

$$\sum_{\mathbf{x} \in X_1} \|\mathbf{w}^T \mathbf{x} - \mathbf{w}^T \boldsymbol{\mu}_1\|_2^2 = \mathbf{w}^T \left(\sum_{\mathbf{x} \in X_1} (\mathbf{x} - \boldsymbol{\mu}_1)(\mathbf{x} - \boldsymbol{\mu}_1)^T \right) \mathbf{w}$$



线性判别分析--算法原理

目标转化为:

$$\max_{\mathbf{w}} J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}} \quad (3.35)$$

注意: $\mathbf{w}$ 成倍
缩放不影响目
标函数的值。

$J(\mathbf{w})$ 称为广义瑞利商(generalized Rayleigh quotient)

因此, **LDA**的目标为最大化广义瑞利商。

线性判别分析--算法原理

目标转化为:

$$\max_{\mathbf{w}} J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}} \quad (3.35)$$

注意: $\mathbf{w}$ 成倍
缩放不影响目
标函数的值。

$J(\mathbf{w})$ 称为广义瑞利商(generalized Rayleigh quotient)

怎么求解上述模型呢?

线性判别分析--算法原理

原问题等价于求解如下问题：

$$\min_{\mathbf{w}} -\mathbf{w}^T \mathbf{S}_b \mathbf{w}, \text{ s. t. } \mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1$$

利用拉格朗日乘子法，

$$L(\mathbf{w}, \lambda) = -\mathbf{w}^T \mathbf{S}_b \mathbf{w} + \lambda(\mathbf{w}^T \mathbf{S}_w \mathbf{w} - 1)$$

$$\nabla_{\mathbf{w}} L(\mathbf{w}, \lambda) = -2\mathbf{S}_b \mathbf{w} + 2\lambda \mathbf{S}_w \mathbf{w} = \mathbf{0}$$

$$\mathbf{S}_b \mathbf{w} = \lambda \mathbf{S}_w \mathbf{w}$$

线性判别分析--算法原理

如果 S_w 可逆，原问题等价于求解如下特征值问题：

$$S_w^{-1} S_b \mathbf{w} = \lambda \mathbf{w}$$

问题 1：如果有多个特征值，选哪一个呢？

问题 2：如何求解特征值？（回忆一下线性代数）

问题 3：如果 S_w 不可逆呢？

问题 4：得到 $\mathbf{w}$ 之后，如何进行分类呢？

线性判别分析--算法原理

问题 1: 如果有多个特征值, 选哪一个呢?

$$\min_{\mathbf{w}} - \mathbf{w}^T \mathbf{S}_b \mathbf{w}, \text{ s. t. } \mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1$$

$$- \mathbf{w}^T \mathbf{S}_b \mathbf{w} = - \lambda \mathbf{w}^T \mathbf{S}_w \mathbf{w} = - \lambda$$

因此, 应取最大特征值对应的特征向量。

线性判别分析--算法原理

问题 2: 如何求解特征值?

$$\mathbf{S}_w^{-1} \mathbf{S}_b \mathbf{w} = \lambda \mathbf{w}$$

令行列式 $|\mathbf{S}_w^{-1} \mathbf{S}_b - \lambda \mathbf{I}| = 0$, 解方程。

线性判别分析--算法原理

问题 3: 如果 S_w 不可逆呢?

正则化思想

$$\min_{\mathbf{w}} -\mathbf{w}^T \mathbf{S}_b \mathbf{w} + \gamma \|\mathbf{w}\|_2^2, \quad \text{s. t.} \quad \mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1$$

线性判别分析--算法原理

问题 4: 得到 $\mathbf{w}$ 之后, 如何进行分类呢?

对降维后的数据采用任一分类器, 如最近邻分类器等

线性判别分析--算法原理

$$J(w) = \frac{w^T S_B w}{w^T S_W w} \quad \leftarrow \text{generalized Rayleigh quotient}$$

$$\begin{aligned} \frac{d}{dw} [J(w)] &= \frac{d}{dw} \left[\frac{w^T S_B w}{w^T S_W w} \right] = 0 \Rightarrow \\ \Rightarrow [w^T S_W w] \frac{d[w^T S_B w]}{dw} - [w^T S_B w] \frac{d[w^T S_W w]}{dw} &= 0 \Rightarrow \\ \Rightarrow [w^T S_W w] 2S_B w - [w^T S_B w] 2S_W w &= 0 \end{aligned}$$

$$\begin{aligned} \frac{[w^T S_W w]}{[w^T S_W w]} S_B w - \frac{[w^T S_B w]}{[w^T S_W w]} S_W w &= 0 \Rightarrow \\ \Rightarrow S_B w - J S_W w &= 0 \Rightarrow \\ \Rightarrow S_W^{-1} S_B w - J w &= 0 \end{aligned}$$

线性判别分析--算法原理

$$S_w^{-1} S_B w = J w \quad \leftarrow \text{特征向量问题}$$

$$S_B w = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T w = (\mu_1 - \mu_2) R$$

$$R = (\mu_1 - \mu_2)^T w \quad \leftarrow \text{scalar}$$

$$J w = S_w^{-1} (S_B w) = S_w^{-1} (\mu_1 - \mu_2) R$$

$$w = \frac{R}{J} S_w^{-1} (\mu_1 - \mu_2)$$



$$w^* = \underset{w}{\operatorname{argmax}} \left\{ \frac{w^T S_B w}{w^T S_w w} \right\} = S_w^{-1} (\mu_1 - \mu_2)$$

LDA: 推广到多类

假定类别数为 N ，样本数为 m ，第 i 类样本集为 X_i

➤ 类内散度矩阵 $\mathbf{S}_w = \sum_{j=1}^N \sum_{\mathbf{x} \in X_i} (\mathbf{x} - \boldsymbol{\mu}_i)(\mathbf{x} - \boldsymbol{\mu}_i)^T$, $\boldsymbol{\mu}_i = \frac{1}{|X_i|} \sum_{\mathbf{x} \in X_i} \mathbf{x}$

➤ 类间散度矩阵 $\mathbf{S}_b = \sum_{i=1}^m m_i (\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^T$ $\boldsymbol{\mu} = \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i$

➤ 全局散度矩阵 $\mathbf{S}_t = \sum_{i=1}^m (\mathbf{x}_i - \boldsymbol{\mu})(\mathbf{x}_i - \boldsymbol{\mu})^T$

LDA: 推广到多类

目标函数为:

$$\max_{\mathbf{W} \in \mathbb{R}^{d \times (N-1)}} J(\mathbf{W}) = \frac{\text{tr}(\mathbf{W}^T \mathbf{S}_b \mathbf{W})}{\text{tr}(\mathbf{W}^T \mathbf{S}_w \mathbf{W})} \quad (3.44)$$

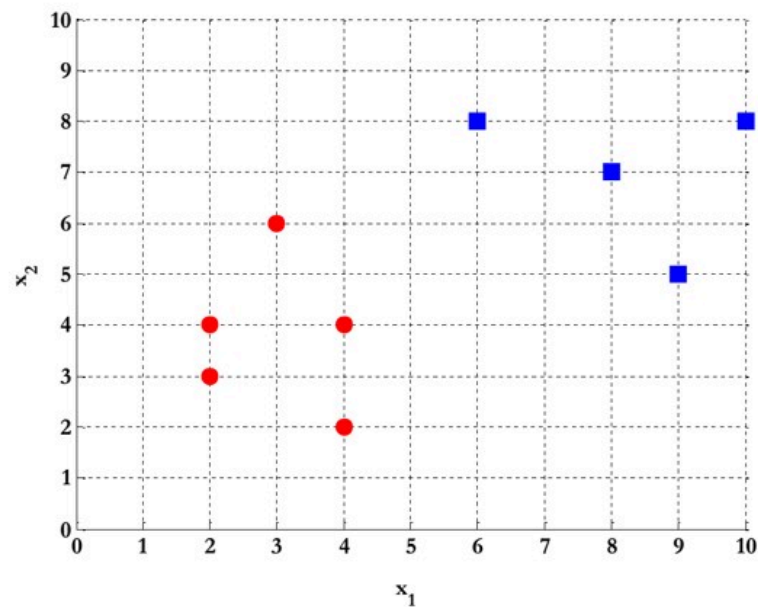
$$\mathbf{S}_b \mathbf{W} = \lambda \mathbf{S}_w \mathbf{W} \quad (3.45)$$

因此, (3.44) 的闭式解为矩阵 $\mathbf{S}_w^{-1} \mathbf{S}_b$ 的 $N - 1$ 个最大广义特征值所对应的特征向量组成的矩阵。

线性判别分析：例子

类别1: (4,2), (2,4), (2,3), (3,6), (4,4)

类别2: (9,10), (6,8), (9,5), (8,7), (10,8)



线性判别分析：例子

➤ 每个类别的均值：

$$\mu_0 = \frac{1}{N_0} \sum_{\mathbf{x} \in X_0} \mathbf{x} = \frac{1}{5} \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} + \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 3 \\ 6 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \end{pmatrix} \right] = \begin{pmatrix} 3 \\ 3.8 \end{pmatrix}$$

$$\mu_1 = \frac{1}{N_1} \sum_{\mathbf{x} \in X_1} \mathbf{x} = \frac{1}{5} \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} + \begin{pmatrix} 6 \\ 8 \end{pmatrix} + \begin{pmatrix} 9 \\ 5 \end{pmatrix} + \begin{pmatrix} 8 \\ 7 \end{pmatrix} + \begin{pmatrix} 10 \\ 8 \end{pmatrix} \right] = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$$

116

线性判别分析：例子

➤ 类别0的协方差矩阵：

$$\begin{aligned}\Sigma_0 &= \frac{1}{N_0} \sum_{\mathbf{x} \in X_0} (\mathbf{x} - \boldsymbol{\mu}_0)(\mathbf{x} - \boldsymbol{\mu}_0)^T = \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 4 \\ 2 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T \\ &\quad + \left[\begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 2 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 2 \\ 3 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T \\ &\quad + \left[\begin{pmatrix} 3 \\ 6 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 3 \\ 6 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 4 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right] \left[\begin{pmatrix} 4 \\ 4 \end{pmatrix} - \begin{pmatrix} 3 \\ 3.8 \end{pmatrix} \right]^T \\ &= \begin{pmatrix} 4 & -1 \\ -1 & 8.8 \end{pmatrix}\end{aligned}$$

117

线性判别分析：例子

➤ 类别1的协方差矩阵：

$$\begin{aligned}\Sigma_1 &= \frac{1}{N_1} \sum_{\mathbf{x} \in X_1} (\mathbf{x} - \boldsymbol{\mu}_1)(\mathbf{x} - \boldsymbol{\mu}_1)^T = \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 9 \\ 10 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &\quad + \left[\begin{pmatrix} 6 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 6 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 9 \\ 5 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 9 \\ 5 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &\quad + \left[\begin{pmatrix} 8 \\ 7 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 8 \\ 7 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T + \left[\begin{pmatrix} 10 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 10 \\ 8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &= \begin{pmatrix} 9.2 & -0.2 \\ -0.2 & 13.2 \end{pmatrix}\end{aligned}$$

118

线性判别分析：例子

➤ 类内散度矩阵(Within-class scatter matrix):

$$\begin{aligned}\mathbf{S}_w &= \mathbf{\Sigma}_0 + \mathbf{\Sigma}_1 \\ &= \begin{pmatrix} 4 & -1 \\ -1 & 8.8 \end{pmatrix} + \begin{pmatrix} 9.2 & -0.2 \\ -0.2 & 13.2 \end{pmatrix} \\ &= 4 \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}\end{aligned}$$

119

线性判别分析：例子

➤ 类内散度矩阵(Within-class scatter matrix):

$$\begin{aligned}\mathbf{S}_b &= (\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)(\boldsymbol{\mu}_0 - \boldsymbol{\mu}_1)^T \\ &= \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right] \left[\begin{pmatrix} 3 \\ 3.8 \end{pmatrix} - \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix} \right]^T \\ &= 4 \begin{pmatrix} 7.29 & 5.13 \\ 5.13 & 3.61 \end{pmatrix}\end{aligned}$$

120

线性判别分析：例子

➤ 求解如下广义特征值问题：

$$S_w^{-1} S_b \mathbf{w} = \lambda \mathbf{w}$$

121

线性判别分析：例子

➤ 求解如下广义特征值问题：

$$|\mathbf{S}_w^{-1}\mathbf{S}_b - \lambda\mathbf{I}| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 3.3 & -0.3 \\ -0.3 & 5.5 \end{pmatrix}^{-1} \begin{pmatrix} 7.29 & 5.13 \\ 5.13 & 3.61 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

$$\Rightarrow \left| \begin{pmatrix} 2.3053 & 1.6223 \\ 1.0585 & 0.7449 \end{pmatrix} - \lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right| = 0$$

线性判别分析：例子

➤ 求解如下广义特征值问题：

$$\Rightarrow \begin{vmatrix} 2.3053 - \lambda & 1.6223 \\ 1.0585 & 0.7449 - \lambda \end{vmatrix} = 0$$

$$\Rightarrow (2.3053 - \lambda)(0.7449 - \lambda) - 1.6223 \times 1.0585 = 0$$

$$\Rightarrow \lambda^2 - 3.0502\lambda = 0$$

$$\Rightarrow \lambda_1 = 0, \quad \lambda_2 = 3.0502.$$

选哪一个
?

线性判别分析：例子

➤ 特征向量：

$$\begin{pmatrix} 2.3053 & 1.6223 \\ 1.0585 & 0.7449 \end{pmatrix} \mathbf{w}_1 = 0 \mathbf{w}_1$$

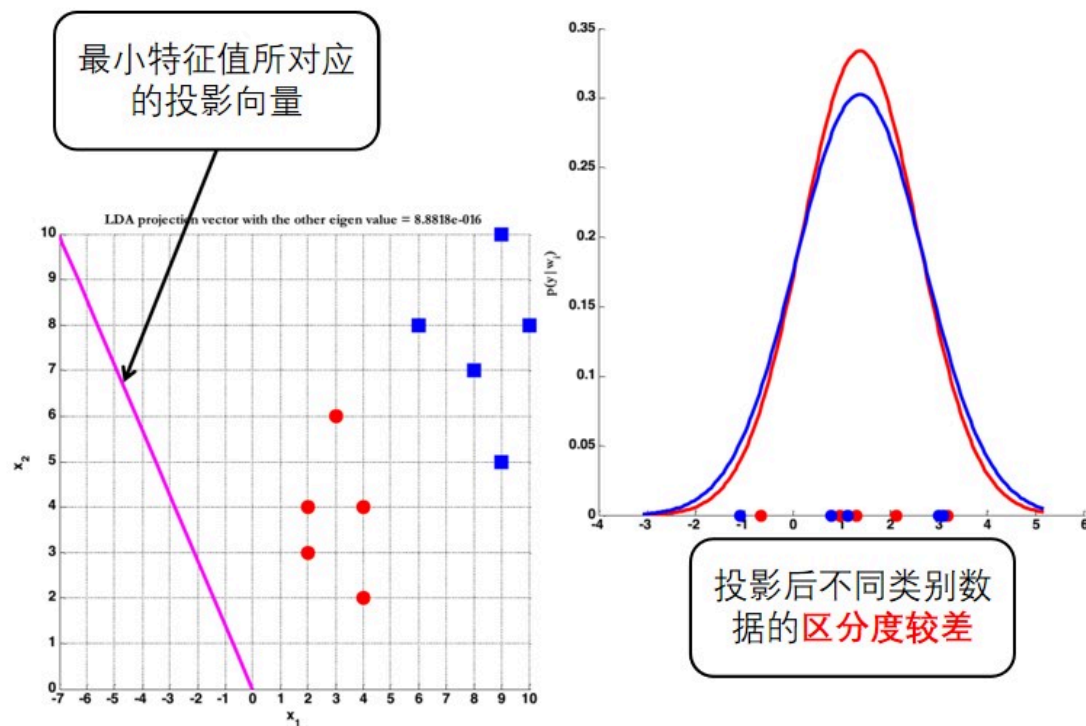
$$\begin{pmatrix} 2.3053 & 1.6223 \\ 1.0585 & 0.7449 \end{pmatrix} \mathbf{w}_2 = 3.0502 \mathbf{w}_2$$

$$\mathbf{w}_1 = \begin{pmatrix} -0.5755 \\ 0.8178 \end{pmatrix} \quad \mathbf{w}_2 = \begin{pmatrix} 0.9088 \\ 0.4173 \end{pmatrix}$$

124

线性判别分析：例子

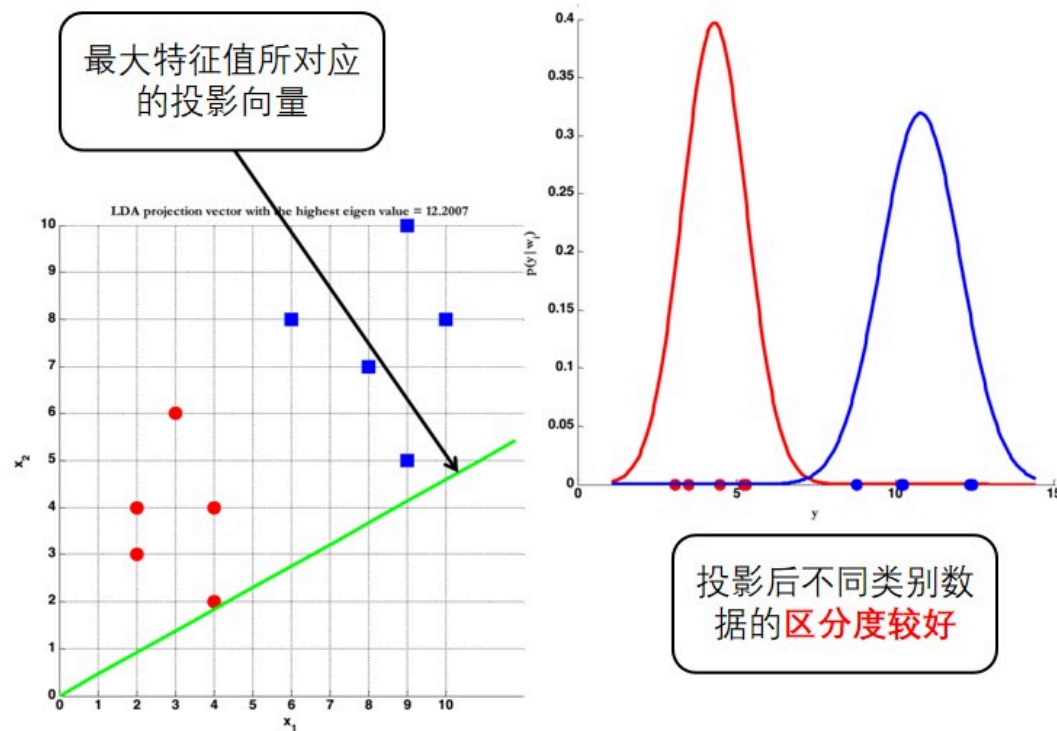
➤ 基于 w_1 的投影：



125

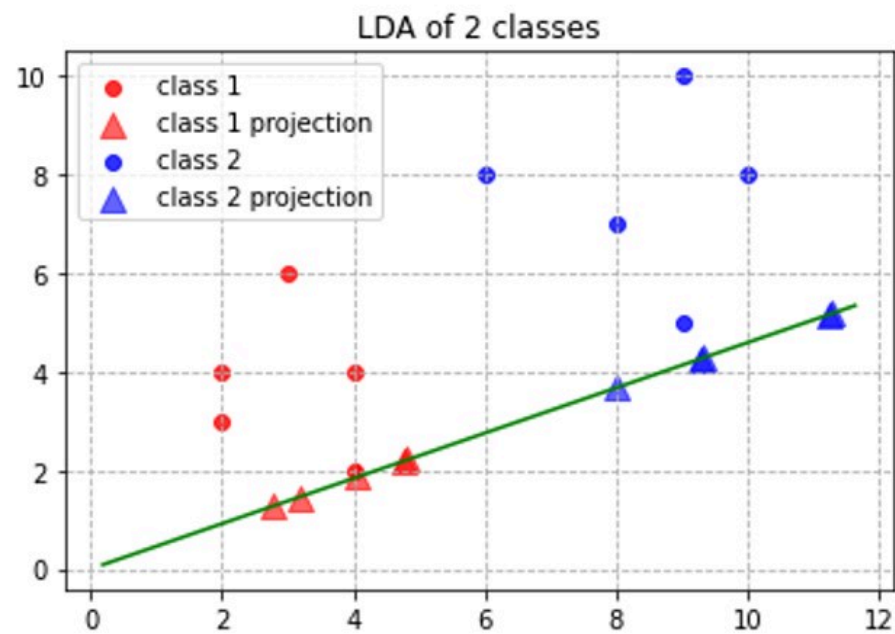
线性判别分析：例子

➤ 基于 w_2 的投影：



126

线性判别分析：例子



127

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

案例：鸢尾花分类

假设你要开发一个自动识别鸢尾花种类的模型，如何利用Linear Discriminant Analysis构建该模型。

1. 读取数据

首先，从sklearn中读取原始数据。

```
# 引入所需要的库文件
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
import matplotlib.pyplot as plt

# 加载sklearn自带的鸢尾花数据集
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names
```

128

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

案例：鸢尾花数据判别分析

假设你要开发一个自动识别鸢尾花种类的模型，如何利用Linear Discriminant Analysis构建该模型。

sklearn.datasets.load_iris

```
sklearn.datasets.load_iris(*, return_X_y=False, as_frame=False)
```

[source]

Load and return the iris dataset (classification).

The iris dataset is a classic and very easy multi-class classification dataset.

Classes	3
Samples per class	50
Samples total	150
Dimensionality	4
Features	real, positive

Read more in the User Guide.

Parameters:

return_X_y : bool, default=False
If True, returns (data, target) instead of a Bunch object. See below for more information about the data and target object.
New in version 0.18.

as_frame : bool, default=False
If True, the data is a pandas DataFrame including columns with appropriate dtypes (numeric). The target is a pandas DataFrame or Series depending on the number of target columns. If return_X_y is True, then (data, target) will be pandas DataFrames or Series as described below.
New in version 0.23.



代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

案例：鸢尾花数据判别分析

```
#查看数据的前2条  
print(X[0:2,:])  
print(y[0:2])  
print(target_names)
```

✓ 0.6s

```
[[5.1 3.5 1.4 0.2]  
 [4.9 3.  1.4 0.2]]  
[0 0]  
['setosa' 'versicolor' 'virginica']
```

130

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

2. 训练模型

利用数据构建线性判别分析模型

```
#利用经验数据构建线性判别分析模型
lda = LinearDiscriminantAnalysis(n_components=2)
X_r = lda.fit(X, y).transform(X)
```

0.8s

```
#查看降维后数据的前2条
print(X_r[0:2,:])
```

✓ 0.2s

```
[[ 8.06179978  0.30042062]
 [ 7.12868772 -0.78666043]]
```

131

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

3. 可视化降维后数据

```
#可视化
plt.figure()

colors = ["navy", "turquoise", "darkorange"]
lw = 2

for color, i, target_name in zip(colors, [0, 1, 2], target_names):
    plt.scatter(
        X_r[y == i, 0], X_r[y == i, 1], alpha=0.8, color=color, label=target_name
    )
plt.legend(loc="best", shadow=False, scatterpoints=1)
plt.title("LDA of IRIS dataset")

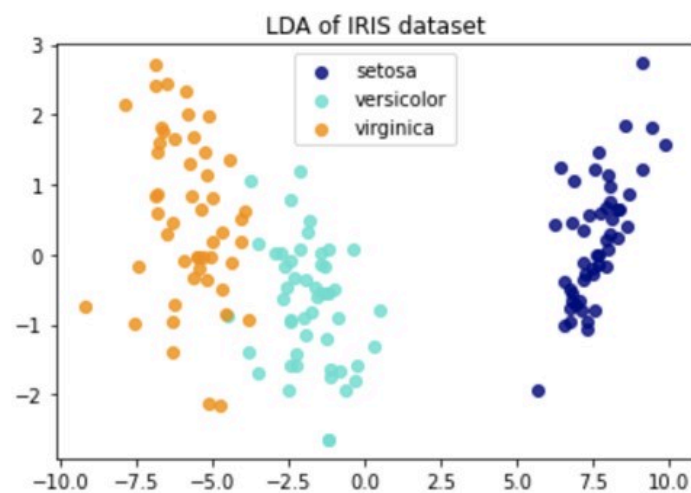
plt.show()
```

132

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

3. 可视化降维后数据



133

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

sklearn.discriminant_analysis.LinearDiscriminantAnalysis

```
class sklearn.discriminant_analysis.LinearDiscriminantAnalysis(solver='svd', shrinkage=None, priors=None,
n_components=None, store_covariance=False, tol=0.0001, covariance_estimator=None) [source]
```

Linear Discriminant Analysis.

A classifier with a linear decision boundary, generated by fitting class conditional densities to the data and using Bayes' rule.

The model fits a Gaussian density to each class, assuming that all classes share the same covariance matrix.

The fitted model can also be used to reduce the dimensionality of the input by projecting it to the most discriminative directions, using the `transform` method.

New in version 0.17: `LinearDiscriminantAnalysis`.

Read more in the User Guide.

Parameters: `solver` : {'svd', 'lsqr', 'eigen'}, default='svd'

Solver to use, possible values:

- 'svd': Singular value decomposition (default). Does not compute the covariance matrix, therefore this solver is recommended for data with a large number of features.
- 'lsqr': Least squares solution. Can be combined with shrinkage or custom covariance estimator.
- 'eigen': Eigenvalue decomposition. Can be combined with shrinkage or custom covariance estimator.

n_components : int, default=None

Number of components ($\leq \min(n_classes - 1, n_features)$) for dimensionality reduction. If None, will be set to $\min(n_classes - 1, n_features)$. This parameter only affects the `transform` method.

134

https://scikit-learn.org/stable/modules/generated/sklearn.discriminant_analysis.LinearDiscriminantAnalysis.html#examples-using-sklearn.discriminant_analysis.LinearDiscriminantAnalysis

代码案例

◆ 线性判别分析使用代码 (采用Scikit-Learn)

```
>>> import numpy as np
>>> from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
>>> X = np.array([[-1, -1], [-2, -1], [-3, -2], [1, 1], [2, 1], [3, 2]])
>>> y = np.array([1, 1, 1, 2, 2, 2])
>>> clf = LinearDiscriminantAnalysis()
>>> clf.fit(X, y)
LinearDiscriminantAnalysis()
>>> print(clf.predict([[-0.8, -1]]))
[1]
```

135

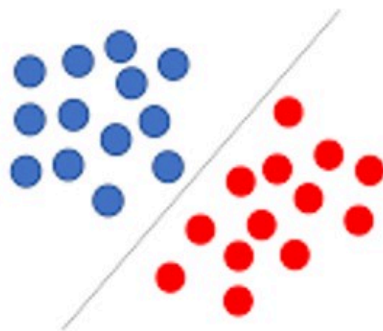


类别不平衡问题

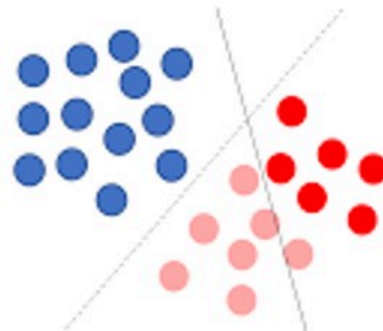
类别不平衡问题

➤ 类别不平衡(Class imbalance):

类别不平衡就是指分类任务中不同类别的训练样例数据差别很大的情况。



类别平衡

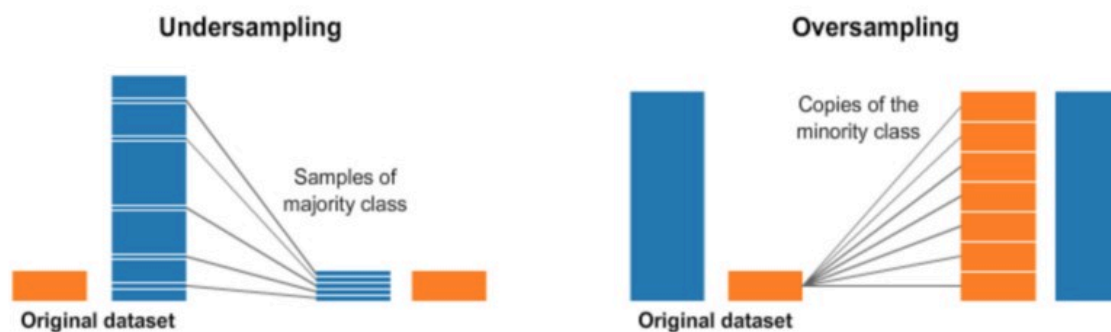


类别不平衡

类别不平衡会对学习过程造成困扰，可能会得到错误的模型。

类别不平衡问题

➤ 欠采样(Undersampling)， 过采样 (Oversampling):



- 欠采样：去除一些反例使得正、反例数目接近，然后再进行学习。
- 过采样：增加一些正例使得正、反例数目接近，然后再进行学习。

谢 谢!